

T18 — Interfacce Funzionali, Espressioni Lambda e Streams API

Predicate, Function, Consumer, Supplier, Method Reference (::) e Pipeline Stream

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

0.1 1. Analisi Teorica Approfondita (Slide T18)

La lezione 18 introduce il paradigma di **programmazione funzionale e dichiarativa** in Java (introdotto in Java 8), imperniato su due pilastri: 1. **Interfacce Funzionali** e **Method Reference (::)**. 2. **Stream API** (`java.util.stream`), per l’elaborazione fluente e parallela di flussi di dati.

0.1.1 1.1 Cos’è una Functional Interface? (Slide 3)

Un’interfaccia si dice **funzionale** se dichiara **esattamente un unico metodo astratto** (pattern noto come **SAM** — *Single Abstract Method*). * **Semantica:** Puramente funzionale; l’esito della chiamata dipende esclusivamente dagli argomenti ricevuti (nessun effetto collaterale occulto). * **L’annotazione `@FunctionalInterface`:** * Non è obbligatoria per far funzionare una lambda, ma è una best practice fondamentale: istruisce il compilatore Java a verificare che l’interfaccia contenga un solo metodo astratto. Se qualcuno tenta di aggiungere un secondo metodo astratto, il compilatore rigetta il codice con errore. * **Metodi esclusi dal conteggio SAM:** * Metodi con implementazione predefinita (`default`). * Metodi statici (`static`). * Metodi pubblici astratti che fanno override di metodi di `java.lang.Object` (es. `boolean equals(Object obj)`).

0.1.2 1.2 Le 4 Interfacce Funzionali Fondamentali (`java.util.function`) (Slide 4-12)

Il package `java.util.function` standardizza i contratti funzionali più frequenti:

Interfaccia	Firma Metodo SAM	Semantica Funzionale	Esempio d’Uso Tipico
<code>Predicate<T></code>	<code>boolean test(T t)</code>	Filtro / Condizione booleana	<code>n -> n % 2 == 0</code>
<code>Function<T, R></code>	<code>R apply(T t)</code>	Trasformazione $T \rightarrow R$	<code>str -> str.length()</code>
<code>Consumer<T></code>	<code>void accept(T t)</code>	Azione con effetto collaterale	<code>str -> System.out.println(str)</code>
<code>Supplier<T></code>	<code>T get()</code>	Generatore / Factory	<code>() -> Math.random()</code>

Varianti e Specializzazioni (Slide 6-12)

1. **Versioni Primitive:** Per evitare l'overhead di boxing/unboxing, Java fornisce interfacce specializzate per `int`, `long`, `double`:

- `IntPredicate` (`boolean test(int v)`), `LongPredicate`, `DoublePredicate`.
- `IntFunction<R>`, `ToIntFunction<T>`, `IntConsumer`, `IntSupplier`.

2. **Varianti a Due Argomenti:**

- `BiPredicate<T, U>`: `boolean test(T t, U u)` (es. `(a, b) -> a > b`).
- `BiFunction<T, U, R>`: `R apply(T t, U u)` (es. `(a, b) -> a + b`).
- `BiConsumer<T, U>`: `void accept(T t, U u)`.

3. **Operatori (Input e Output dello stesso tipo):**

- `UnaryOperator<T>` extends `Function<T, T>`: riceve un `T` e restituisce un `T`.
- `BinaryOperator<T>` extends `BiFunction<T, T, T>`: riceve due `T` e restituisce un `T`.

0.1.3 1.3 I Quattro Tipi di Method Reference (::) (Slide 13-15)

Il *Method Reference* è uno zucchero sintattico introdotto in Java 8 per rendere le espressioni lambda ancora più concise quando queste si limitano a inoltrare direttamente i propri parametri a un metodo esistente:

Categoria	Sintassi Method Reference	Equivalente Lambda Esplicita
1. Metodo Statico	<code>Math::randomInteger::parseInt</code>	<code>() -> Math.random()str -> Integer.parseInt(str)</code>
2. Metodo d'Istanza su Oggetto Esistente (Bound)	<code>System.out::printlnhexDigits::charAt</code>	<code>x -> System.out.println(x)i -> hexDigits.charAt(i)</code>
3. Metodo d'Istanza su Tipo Arbitrario (Unbound)	<code>String::toUpperCaseString::length</code>	<code>str -> str.toUpperCase()str -> str.length()</code>
4. Costruttore	<code>Person::newInteger[]::new</code>	<code>str -> new Person(str)size -> new Integer[size]</code>

0.1.4 1.4 La Stream API (java.util.stream) (Slide 16-18)

Uno `Stream<T>` è una **sequenza di elementi generata da una sorgente** che supporta operazioni aggregate di calcolo.

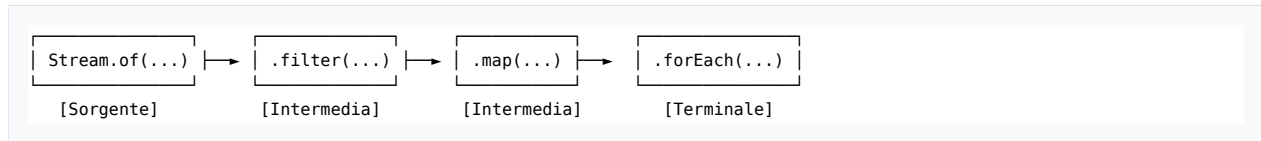
Le Tre Proprietà Chiave di uno Stream:

1. **Pipelining:** Le operazioni intermedie restituiscono un nuovo stream, permettendo la concatenazione fluente (*method chaining*).
2. **Internal Iteration:** A differenza delle collezioni dove lo sviluppatore controlla l'iterazione tramite `for/while` (*iterazione esterna*), lo stream gestisce il ciclo internamente. Ciò consente alla JVM di ottimizzare l'esecuzione e abilitare il parallelismo trasparente (`parallelStream()`).
3. **Lazy Evaluation (Valutazione Pigra):** Le operazioni intermedie non vengono eseguite quando vengono dichiarate; restano "in attesa". L'intera pipeline viene elaborata in un unico passaggio solo quando viene invocata un'operazione terminale!

0.1.5 1.5 Ciclo di Vita di una Pipeline Stream (Slide 18-24)

Una pipeline si compone tassativamente di 3 fasi:

Sorgente (Source) → Zero o più Operazioni Intermedie → Una Operazione Terminale



1. Creazione della Sorgente (Slide 19, 21-22)

- Da array: `Arrays.stream(arr)` o `Stream.of(v1, v2, v3)`.
- Da collezione: `list.stream()` o `set.stream()`.
- **Generazione infinita:**
 - `Stream.iterate(seed, UnaryOperator)`: applica iterativamente l'operatore a partire dal seme. Es: `Stream.iterate(0, i -> i + 1)` genera 0, 1, 2, 3, ...
 - `Stream.generate(Supplier)`: invoca continuamente il fornitore. Es: `Stream.generate(Math::random)`.
 - Per evitare loop infiniti, si arresta la generazione con l'operazione intermedia `.limit(n)`.

2. Operazioni Intermedie (Slide 23, 25-29, 34-36) Restituiscono un nuovo `Stream<R>` e sono pigre (*lazy*):
* `filter(Predicate<T>)`: trattiene solo gli elementi per cui il predicato è true.
* `map(Function<T, R>)`: trasforma ciascun elemento $T \rightarrow R$ (mappatura $1 \rightarrow 1$).
* `flatMap(Function<T, Stream<R>>)`: appiattisce strutture annidate (es. matrici `Integer[][]` o liste di liste) in un unico stream monodimensionale.
* `distinct()`: elimina i duplicati (in base a `equals` e `hashCode`).
* `sorted()` / `sorted(Comparator)`: ordina gli elementi.
* `limit(n)`: tronca lo stream ai primi n elementi.
* `skip(n)`: scarta i primi n elementi.

3. Operazioni Terminali (Slide 20, 23, 30-33) Consumano lo stream e producono un risultato o un effetto collaterale:
* **Chiusura su Collezioni / Array (Slide 20)**:
* `stream.toList()`: raccoglie gli elementi in una `List` immutabile (Java 16+).
* `stream.collect(Collectors.toSet())`: raccoglie in un `Set`.
* `stream.toArray(Integer[]::new)`: esporta in array tipizzato.
* **Iterazione / Consumo**: `stream.forEach(Consumer<T>)`.
* **Conteggio / Riduzione**: `stream.count()`, `stream.reduce(...)`.
* **Matching**: `anyMatch(Predicate)`, `allMatch(Predicate)`, `noneMatch(Predicate)`.
* **Ricerca & optional<T> (Slide 31-32)**:
* `findFirst()`, `findAny()`: restituiscono un `Optional<T>`, contenitore sicuro introdotto per debellare le `NullPointerException`.
* Metodi di `Optional`: `isPresent()`, `get()`, `orElse(valoreDefault)`.

Attenzione / Trappola d'Esame

Regola Aurea degli Stream (Slide 24 — Domanda d'Esame): Uno Stream può essere consumato UNA SOLA VOLTA! Se si tenta di invocare una seconda operazione terminale sullo stesso oggetto Stream, la JVM solleva a runtime: `java.lang.IllegalStateException: stream has already been operated upon or closed`.

0.2 2. Analisi Dettagliata del Codice (Lezione17/MavenDate)

In `Lezione17/MavenDate`, il prof. Pasqua mantiene l'intera architettura pregressa e aggiunge il file `Main-Stream.java`, che esemplifica in modo magistrale tutte le funzionalità degli Stream appena studiate.

0.2.1 2.1 Analisi Riga per Riga di MainStream.java

```
1 package it.oop.ui;
2
3 import it.oop.core.AmericanDate;
4 import it.oop.core.FormattedDate;
5 import it.oop.core.ItalianDate;
6
7 import java.util.List;
8 import java.util.stream.Stream;
9 import it.oop.core.Date;
10 import static java.util.stream.Stream.generate;
11
12 public class MainStream {
13     public static void main(String[] args) {
14         // ESEMPIO 1: Generazione con iterate, filtro e limit
15         // Sequenza generata: "a", "aa", "aaa", "aaaa", ...
16         Stream<String> stream = Stream.iterate("a", s -> s + "a")
17             .filter(s -> s.length() % 2 == 1) // Tiene solo stringhe di lunghezza DISPARI
18             .limit(10);                       // Si arresta dopo le prime 10 stringhe dispari
19         List<String> list = stream.toList();    // Operazione terminale: unwrap a List
20         System.out.println(list.toString());
21
22         // ESEMPIO 2: Generazione con generate, Supplier e test di primalità
23         List<Integer> list2 = Stream.generate(() -> (int) (Math.random() * 20)) // Generatore
24             .infinite() [0..19]
25             .filter(i -> isPrime(i)) // Filtra numeri
26             .limit(10) // Ne estrae
27             .toList(); // Operazione
28             terminale
29         System.out.println(list2.toString());
30
31         // ESEMPIO 3: Pipeline su oggetti di dominio del progetto (Date)
32         Stream.of(new ItalianDate(15,1,2025), new Date(16,1,2025), new ItalianDate(2,2,2025))
33             .filter(d -> d instanceof ItalianDate) // Filtra escludendo le Date generiche
34             .map(d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear())) // Converte in
35             AmericanDate
36             .forEach(System.out::println); // Method reference per stampa su stdout
37     }
38
39     // Algoritmo di primalità implementato interamente con Stream!
40     private static boolean isPrime(int n) {
41         return n == 0 ? false : Stream.iterate(1, i -> i + 1)
42             .limit(n) // Genera i numeri da 1 a n
43             .map(i -> n % i) // Calcola il resto della divisione (0 indica divisore esatto)
44             .filter(i -> i == 0) // Mantiene solo i resti nulli (i divisori)
45             .count() <= 2; // Se i divisori sono <= 2 (cioè 1 e se stesso), è primo!
46     }
47 }
```

0.3 3. Risultato di Compilazione ed Esecuzione Reale

Comando eseguito nel progetto [Lezione17/MavenDate](#):

```
1 mvn compile exec:java -Dexec.mainClass="it.oop.ui.MainStream"
```

Output ottenuto:

```
1 [a, aaa, aaaaa, aaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa, aaaaaaaa]
```

```
2 [7, 11, 13, 19, 3, 13, 17, 5, 19, 1]
3 1/15/2025
4 2/2/2025
```

0.3.1 Decodifica dell'Output:

1. [a, aaa, aaaaa, ..., aaaaaaaaaaaaaaaaaa] → Lista delle prime 10 stringhe a lunghezza dispari (1, 3, 5, 7, 9, 11, 13, 15, 17, 19 caratteri).
2. [7, 11, 13, 19, 3, 13, 17, 5, 19, 1] → Dieci numeri primi casuali estratti nell'intervallo [0, 20) e validati dalla pipeline stream di `isPrime(int n)`.
3. 1/15/2025 e 2/2/2025 → Le due istanze di `ItalianDate` (15/1/2025 e 2/2/2025) superano il `filter(instanceof ItalianDate)`, vengono convertite tramite `map` in `AmericanDate` (formato M/D/Y) e stampate a video tramite `System.out::println`. L'istanza generica `new Date(16,1,2025)` viene invece scartata dal filtro.

Dimmi “vai” per procedere con il **Blocco 18** (Lezione 18 / Slide T19 — *Documentation and Unit Testing*, Javadoc, JUnit 3 vs JUnit 4 vs JUnit 5, asserzioni, ciclo di vita dei test `@BeforeEach/@AfterEach`, e l'analisi di `TestDate.java` e `TestItalianDate.java` in *Lezione18/MavenDate*).